

Step-by-Step Implementation: Migrating to Azure ML Compute Cluster

This document provides a technical walkthrough for transitioning your `house-price` project from a Compute Instance to a Compute Cluster, including cost calculations for your specific VM size and UI-based steps for the migration.

1. Cost Analysis: Standard_F8s_v2

You are using the **Standard_F8s_v2** (8 Cores, 16 GB RAM). Below is the cost breakdown for your team of 20 data scientists.

Comparison Table

Metric	20 Compute Instances (Always On)	1 Compute Cluster (Autoscale 0-20)
Hourly Rate (Est.)	~\$0.34 per node <code>1</code>	~\$0.34 per active node
Daily Cost (8h)	~\$54.40 (Fixed)	~\$54.40 (Only if all 20 run for 8h)
Idle Cost (Night)	~\$108.80 (16h)	\$0.00
Monthly Total	~\$4,800+	~\$1,200 - \$1,600 (Est. actual usage)

Worst-Case Concurrent Scenario

If all 20 data scientists trigger their pipeline at the exact same time:

- Scaling:** The cluster detects 20 pending jobs and provisions 20 nodes of `Standard_F8s_v2`.
- Peak Hourly Burn:** $\$0.34 \times 20 = \6.80 per hour.
- Completion:** Once the jobs finish (e.g., in 15 minutes), the cluster scales to 0.
- Actual Cost for that run:** $\$6.80 / 4 = \1.70 total for all 20 scientists to finish their work.

2. UI-Based Implementation (Step-by-Step)

Step A: Create the Cluster (Studio UI)

1. Open [Azure ML Studio](#).
2. Go to **Manage** -> **Compute** -> **Compute clusters** -> **+ New**.
3. **Select VM Size:** Search for `Standard_F8s_v2` . Click **Next**.
4. **Settings:**
 - **Compute name:** `shared-training-cluster`
 - **Min nodes:** `0` (Critical for saving money)
 - **Max nodes:** `20`
 - **Idle seconds before scale down:** `120`
5. **Advanced Settings:** Ensure **Managed Identity** is enabled (System-assigned). This allows the cluster to pull from MCR without root access.
6. Click **Create**.

Step B: Create the Environment (Studio UI)

1. Go to **Assets** -> **Environments** -> **Custom environments** -> **+ Create**.
2. **Name:** `kedro-house-price-env` .
3. **Select source:** "Upload existing directory" (Upload your `house-price` folder containing `requirements.txt`).
4. **Environment type:** Select "Conda with Docker image".
5. **Docker settings:** Use the default MCR image: `mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04` .
6. **Conda settings:** The UI will read your `requirements.txt` .
7. Click **Next** and **Create**. Azure ML will now build the Docker image in the background.

3. Code Implementation: The "No-Root" Fix

To run the pipeline from your current notebook without needing root access, you will submit a "Job".

Create `submit_job.py`

Place this file in your project folder. It tells Azure ML to take your code, use the cluster, and use the pre-built environment.

```
Python
```

```
from azure.ai.ml import MLClient, command
from azure.identity import DefaultAzureCredential

# 1. Connect to your workspace
ml_client = MLClient.from_config(credential=DefaultAzureCredential())

# 2. Define the job
job = command(
    code="./", # Uploads your house-price code
    command="python -m kedro run",
    environment="kedro-house-price-env@latest",
    compute="shared-training-cluster",
    experiment_name="house_price_migration",
    display_name="First Cluster Run"
)

# 3. Submit
returned_job = ml_client.jobs.create_or_update(job)
print(f"Job submitted! Monitor here: {returned_job.studio_url}")
```

4. Summary of Changes

1. **For You:** You no longer type `kedro run` in the terminal. You run the `submit_job.py` script.
2. **For the Manager:** The "Learning Task" problem is solved because the Cluster's Managed Identity pulls the Docker images, bypassing the need for user-level root access.
3. **For the Budget:** You stop paying for 20 individual VMs. You only pay for what is actually computing.

References

- [1] Azure VM Pricing (Fsv2-series):
- [2] Azure ML Compute Cluster Autoscale: